Task 2

Intelligent Multi-Class Natural Language Text Sentiment Classifier

AI Internship Report | Name: FIZA ASGHAR | Date: 09 OCTOBER 2026

1. Objective

To build an NLP classifier that maps unstructured text statements to one of several sentiment classes (negative, neutral, positive). The task requires Python (Scikit-Learn or PyTorch), text preprocessing with stop-word removal and lemmatization, conversion of text to numerical form using TF-IDF or embedding models, and evaluation using the F1-score.

2. Summary of Results

Three solutions were developed and compared on the same task. Performance improved at each stage, with the fine-tuned transformer achieving the best Macro F1 of 0.822.

Model	Features	Accuracy	Macro F1
v1: Logistic Regression	TF-IDF (word 1-2 grams)	77.2%	0.725
v2: Linear SVM (tuned)	TF-IDF (word + char)	82.0%	0.770
v3: RoBERTa (fine-tuned)	**Transformer embeddings**	**86.0%**	**0.822**

3. Dataset

The Twitter US Airline Sentiment dataset (Kaggle) was used. After removing missing values and duplicate tweets, 14,427 tweets remained in three classes:

Class	Tweets	Share
Negative	9,080	62.9%
Neutral	3,057	21.2%
Positive	2,290	15.9%

The dataset is imbalanced (negative tweets dominate). Therefore Macro F1, which averages the F1-score of every class equally, was used as the main metric, and class weighting was applied during training. An 80/20 stratified train/test split (random_state = 42) kept the class proportions identical in both sets.

4. Methodology

4.1 Preprocessing (v1 and v2)

- Lowercasing; removal of URLs, @mentions, hashtag symbols, punctuation, numbers and emojis.
- Stop-word removal using the NLTK English list. Negation words (not, no, never, nor) were deliberately kept because removing them can flip the meaning of a sentence.
- Lemmatization using the NLTK WordNet lemmatizer (noun and verb forms), e.g. "delayed" becomes "delay".
- v2 only: contraction expansion (don't becomes do not) and negation handling, where the word after a negation is marked ("not good" becomes "not_good").

4.2 Feature extraction

- **v1:** TF-IDF on word unigrams and bigrams (max 20,000 features).
- **v2:** word TF-IDF (1-2 grams) combined with character TF-IDF (2-5 grams, char_wb) using sublinear term frequency. Character n-grams capture spelling variations and slang common in tweets.
- **v3:** contextual embeddings learned by a transformer (RoBERTa). Stop-word removal and lemmatization were not applied here because transformers need the complete sentence structure to interpret context and negation.

4.3 Models and training

- **v1:** Logistic Regression and Linear SVM with balanced class weights; the better one was selected on Macro F1.
- **v2:** GridSearchCV (3-fold, scoring = Macro F1) over Logistic Regression (C = 1, 3, 10) and Linear SVM (C = 0.1, 0.3, 1). Best: Linear SVM, C = 0.1 (cross-validation Macro F1 = 0.736).
- **v3:** fine-tuning of the pre-trained model cardiffnlp/twitter-roberta-base-sentiment-latest in PyTorch (Hugging Face Transformers). This model was pre-trained on a large tweet corpus and had already been trained for general tweet sentiment, so this step is transfer learning: it was further fine-tuned on the airline data. Settings: max length 64, batch size 32, 3 epochs, AdamW (learning rate 2e-5, weight decay 0.01), linear warm-up (10%), class-weighted cross-entropy loss, mixed precision (fp16) on a Tesla T4 GPU. 10% of the training data was held out as a validation set and the epoch with the best validation Macro F1 was kept (epoch 3, validation Macro F1 = 0.847). Training took about 2 minutes.

5. Detailed Results

5.1 Per-class F1-score on the test set

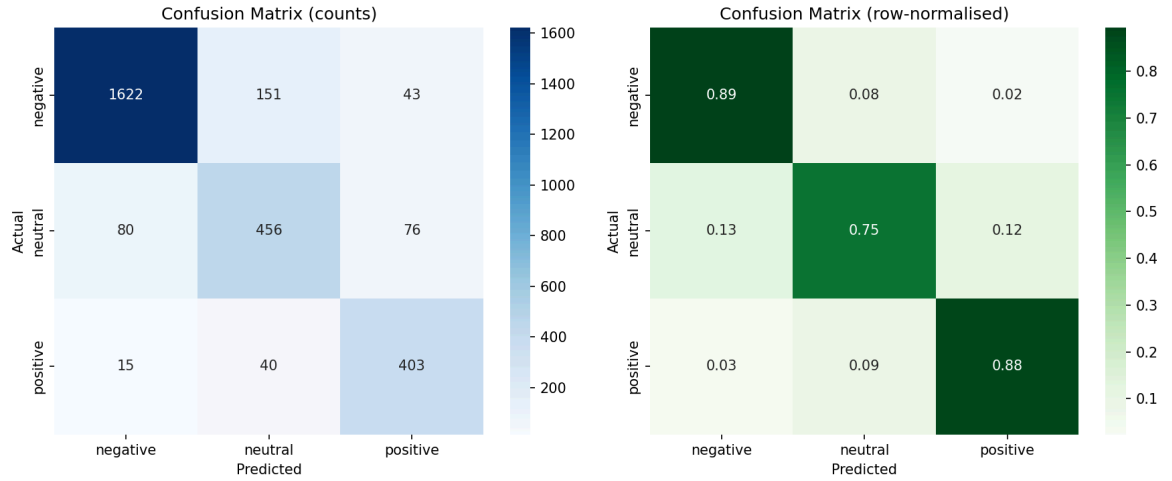
Class	v1 LogReg	v2 SVM	v3 RoBERTa
Negative	0.850	0.887	0.918
Neutral	0.600	0.662	0.724
Positive	0.724	0.760	0.822
Macro average	**0.725**	**0.770**	**0.822**

5.2 Final model (v3) classification report

Class	Precision	Recall	F1-score	Support
Negative	0.945	0.893	0.918	1816
Neutral	0.705	0.745	0.724	612
Positive	0.772	0.880	0.822	458
Accuracy			0.860	2886
Macro average	0.807	0.839	0.822	2886

5.3 Confusion matrix (v3)

Actual \ Predicted	Negative	Neutral	Positive
Negative	1622	151	43
Neutral	80	456	76
Positive	15	40	403



6. Analysis and Discussion

- Accuracy rose from 77.2% to 86.0% and Macro F1 from 0.725 to 0.822 across the three versions. All three classes improved in every step.
- Negative sentiment is predicted best (F1 0.918), which is expected because it has by far the most training examples.
- Neutral is the hardest class (F1 0.724). The confusion matrix shows that most errors involve neutral tweets: 80 neutral tweets were predicted negative and 76 positive, and 151 negative tweets were predicted neutral. Neutral text has few strong sentiment words, so its boundary with the other classes is blurry; some tweets are also ambiguous even for human annotators.
- The positive class has high recall (0.880) but lower precision (0.772), meaning the model sometimes labels neutral tweets as positive.

- The improvement from v1 to v2 shows the value of better features (negation handling, character n-grams) and hyper-parameter tuning. The further improvement in v3 comes from contextual embeddings, which understand word meaning in context (for example sarcasm and negation) better than TF-IDF, which only counts words.

7. Limitations

- Domain bias: the models were trained only on airline tweets. A neutral sentence such as "The package arrived on Tuesday" was classified as negative by the v2 model because words such as "arrived" and "delay" mostly occur in complaints in this dataset. Performance on other domains (product reviews, news) would need re-training or new data.
- The three versions use slightly different test sets (2,886 tweets for v1 and v3; 2,881 for v2, because tweets that became empty after cleaning were dropped), so the comparison is close but not perfectly identical.
- The v3 base model had already seen tweet sentiment data before fine-tuning, which is part of why it performs strongly. A plain BERT model without this prior training would likely score lower.
- Results come from a single train/test split; repeating with different random seeds would give a measure of variance.

8. Conclusion

A complete multi-class sentiment classification pipeline was built in Python. Starting from a TF-IDF and Logistic Regression baseline (Macro F1 0.725), systematic improvements in preprocessing, features and tuning raised performance to 0.770, and fine-tuning a transformer model achieved the best result of Macro F1 0.822 with 86.0% accuracy. Future work could include collecting more neutral examples, data augmentation, ensemble models, and testing on other domains.

9. Deliverables

- sentiment_classifier.py (v1), sentiment_classifier_v2.py (v2), task2_bert_colab.ipynb (v3, Google Colab)
- confusion_matrix.png, confusion_matrix_v2.png, confusion_matrix_bert.png
- misclassified_examples.csv and misclassified_examples_bert.csv (error analysis), sentiment_model.joblib (saved v2 model)